Chapter 8

Accessing Pod Metadata

& Other Resources

Pod 안의 앱이 자기 자신과 클러스터를 아는 법

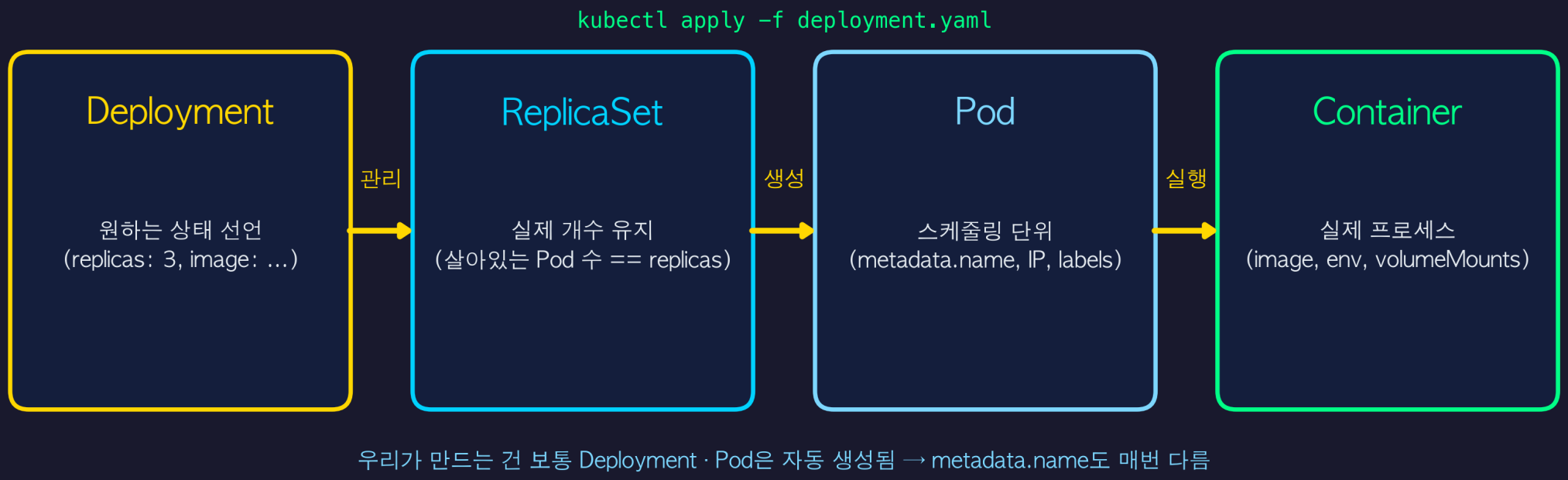


오늘의 두 축

알고 싶은 것	메커니즘
자기 자신의 메타데이터 (Pod name, IP, labels…)	**Downward API** env / volume 주입
다른 리소스 (Pod 목록, Service…)	**API 서버 직접 호출** ServiceAccount 토큰

> 핵심 차이: Downward API는 **외부 통신 없음**, API 호출은 **HTTPS 통신**.

잠깐, 우리는 Pod을 직접 만들지 않는다



우리가 만드는 건 보통 Deployment · Pod은 자동 생성됨 → metadata.name도 매번 다름

- 보통 작성하는 건 `Deployment` YAML 한 장
- Pod은 ReplicaSet이 자동 생성 → **이름·IP는 매번 다름**
- 그래서 앱이 실행 시점에 자기 메타데이터를 알아야 함

1. Downward API

Pod가 자기 자신을 아는 법



왜 Downward API가 필요한가?

```
[로깅] "POST /order" 요청 처리 – 어느 Pod이 처리했지?
[메트릭] 이 컨테이너의 CPU limit은 얼마지?
[클러스터링] 내 IP는 뭐고, 같이 떠있는 Pod IP들은?
```

- 앱 코드는 `metadata.name`을 **모른다**
- k8s API 호출하기엔 **너무 무거움**
- → **kubelet이 알려준다** (env 또는 파일로 주입)

노출 방법 두 가지

방식	어디에 정의	특징
환경 변수	`containers[].env[].valueFrom`	시작 시 1회 설정, 변경 시 재시작 필요
downwardAPI 볼륨	`volumes[].downwardAPI.items[]`	파일 1개 = 필드 1개, **런타임 갱신 가능**

> 주요 차이: **변경 반영 시점**과 **표현 가능한 범위**.

fieldRef: 노출 가능한 필드

둘 다 가능 (env / volume):

- `metadata.name`, `metadata.namespace`, `metadata.uid`
- `metadata.labels['<KEY>']`, `metadata.annotations['<KEY>']` (단일 키)

env 전용 (volume `fieldRef` 불가):

- `spec.serviceAccountName`, `spec.nodeName`
- `status.hostIP`, `status.hostIPs`, `status.podIP`, `status.podIPs`

volume 전용 (env 불가):

- `metadata.labels` (전체), `metadata.annotations` (전체)



resourceFieldRef: 컨테이너 리소스

```yaml
env:
- name: CPU_REQUEST_MILLICORES
  valueFrom:
    resourceFieldRef:
      resource: requests.cpu
      divisor: 1m                # "100m" → "100"
- name: MEMORY_LIMIT_MIB
  valueFrom:
    resourceFieldRef:
      resource: limits.memory
      divisor: 1Mi
```

가능한 리소스: `cpu`, `memory`, `ephemeral-storage`, `hugepages-*`
(각각 `requests` / `limits`)

> ⚠ `limits` 미설정 시 → **node allocatable 최대값**으로 채움 (자세한 limits 동작은 ch14)

env로 주입하는 예시

```yaml
env:
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name
- name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
- name: NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
```

→ 컨테이너 안: `echo $POD_NAME` → `kubia-abc123`



볼륨으로 주입 (multi-line 노출)

```yaml
volumeMounts:
- name: downward
  mountPath: /etc/downward
volumes:
- name: downward
  downwardAPI:
    items:
    - path: podName
      fieldRef:
        fieldPath: metadata.name
    - path: labels                  # ← env로는 불가
      fieldRef:
        fieldPath: metadata.labels
```

→ `/etc/downward/podName`, `/etc/downward/labels` 파일 생성



핵심 차이: 런타임 갱신

방식	labels 변경 시	리소스 리사이즈 시
env	갱신 안 됨 (재시작 필요)	갱신 안 됨 (재시작 필요)
volume	**자동 갱신** ✅	**자동 갱신** ✅ (k8s 1.35+ stable)

```
# 실험: labels 추가
kubectl label pod kubia env=production

# 컨테이너 안에서 5초 후
$ cat /etc/downward/labels
env="production"
app="kubia"
```

두 메커니즘, 같은 작업 — 비용 비교

Downward API

읽기 1줄

```
os.Getenv("POD_NAME")
```

통신 0회

kubelet이 이미 주입

인증 불필요

TLS · 토큰 · RBAC 없음

지연 ≈ 0

"내" 정보면 충분 → 이 길로

API Server 호출

읽기 5+줄

```
token = open(...).read()
ca = "/var/run/.../ca.crt"
headers = {Authorization: ...}
requests.get(url, verify=ca)
```

통신 1회

Pod → kube-proxy → apiserver

인증 필수

TLS · Bearer · RBAC

지연 ≈ 수십 ms (네트워크)

"남"의 정보 필요할 때만

> Downward API로 끝낼 수 있으면 **항상** Downward API. API 호출은 "남의 정보"가 꼭 필요할 때만.

2. Kubernetes API 직접 접근

클러스터 안의 다른 리소스 조회



왜 API 서버 직접 호출?

```
Downward API: 자기 자신의 메타데이터만 ✅

질문:
- 같은 Service의 다른 Pod IP는?  ❌
- 클러스터의 모든 Service 목록은?  ❌
- 최근 발생한 이벤트는?  ❌

→ 모두 API Server에 쿼리 필요
```



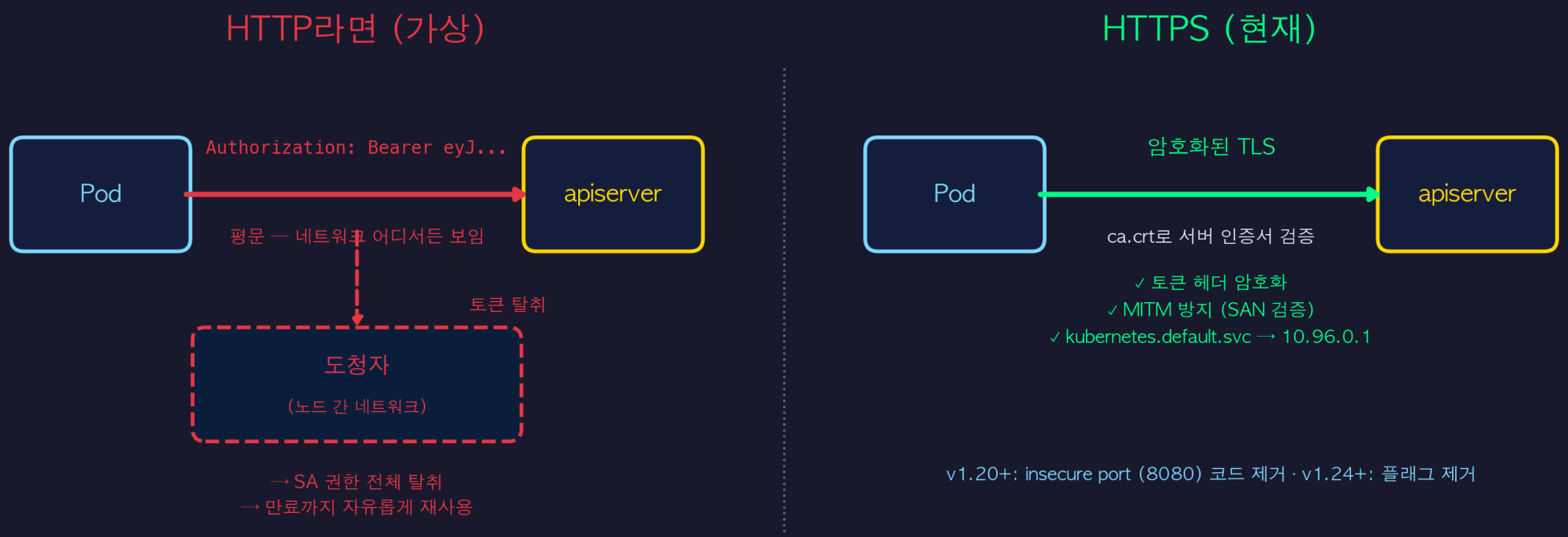
접근 경로



모든 Pod에서 도달 가능 · DNS는 모든 클러스터에서 동일

- `kubernetes.default.svc` Service는 **모든 클러스터에 항상 존재**
- ClusterIP는 클러스터마다 다르지만 DNS는 동일
- 모든 Pod에서 자동으로 도달 가능

왜 HTTPS인가? — Bearer token의 운명



- Bearer token = 평문이면 캡처 즉시 **SA 권한 전체 탈취**
- HTTPS = 토큰 암호화 + `ca.crt` 로 서버 인증서 SAN 검증 (MITM 차단)
- v1.20+ 부터 평문 8080 포트 **코드에서 제거**

인증: ServiceAccount

모든 Pod에 **자동 마운트**되는 경로:

```
/var/run/secrets/kubernetes.io/serviceaccount/
├── ca.crt        # apiserver TLS 검증용 CA 인증서
├── namespace     # 현재 네임스페이스
└── token         # JWT Bearer token
```

- 명시 안 하면 `default` ServiceAccount 사용
- 별도 SA 만들어 권한 분리 권장



호출 예시

```
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
NS=$(cat /var/run/secrets/kubernetes.io/serviceaccount/namespace)
CACERT=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt

curl --cacert $CACERT \
  -H "Authorization: Bearer $TOKEN" \
  https://kubernetes.default.svc/api/v1/namespaces/$NS/pods
```

→ 인증은 통과! 하지만…

```
{ "kind": "Status", "status": "Failure",
  "message": "pods is forbidden: ...", "code": 403 }
```



RBAC 벽 (k8s 1.6+ 기본 활성)

```
default ServiceAccount의 권한: ❌ 없음
```

해결: Role + RoleBinding으로 권한 부여

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader }
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: pod-reader-binding }
subjects:
- kind: ServiceAccount
  name: default
```

실습: 5단계 검증 시나리오

1. SA 토큰이 마운트되었는지 확인
2. **인증 없이** 호출 → 401 거부
3. **토큰으로** 호출 → 401 통과, 그러나 403 forbidden (RBAC)
4. **Role + RoleBinding** 부여 → pods 허용 ✅
5. 같은 토큰으로 services 호출 → **여전히 403** (Role 범위 밖)

> 5단계의 의미: **권한이 정확히 부여된 리소스에만 적용된다**. 과허용 없음.

3. Ambassador 패턴

인증 로직을 사이드카에 위임



문제: 앱 코드에 인증 로직?

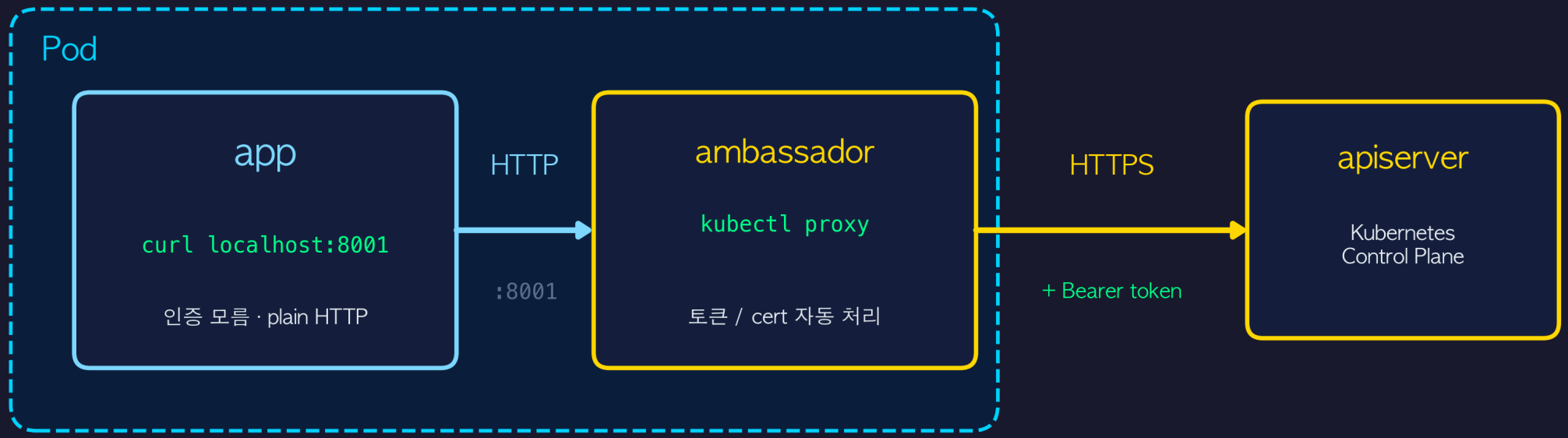
```
# 앱 코드에 이런 거 다 들어가야 함:
token = open("/var/run/.../token").read()
ca = "/var/run/.../ca.crt"
ns = open("/var/run/.../namespace").read()

requests.get(
  f"https://kubernetes.default.svc/api/v1/namespaces/{ns}/pods",
  headers={"Authorization": f"Bearer {token}"},
  verify=ca
)
```

→ 모든 언어, 모든 앱마다 반복 ❌



해법: kubectl proxy 사이드카



- `app` : `curl localhost:8001/api/...` (plain HTTP)
- `proxy` : 토큰/cert 자동 처리

Pod 매니페스트

```yaml
spec:
  containers:
  - name: main
    image: curlimages/curl:latest      # ← tutum/curl 대체
    command: ["sleep", "9999999"]
  - name: ambassador
    image: bitnami/kubectl:latest      # ← luksa/kubectl-proxy:1.6.2 대체
    command: ["kubectl", "proxy", "--port=8001"]
```

→ `main` 에서 `curl localhost:8001/api/v1/.../pods` ✅

클라이언트 라이브러리가 더 깔끔

언어	패키지
Go	`k8s.io/client-go`
Python	`kubernetes` (pip)
Java	`io.kubernetes:client-java`
Node.js	`@kubernetes/client-node`

- **InClusterConfig**: SA 토큰 자동 로드
- **타입 안전**: Pod struct 등
- **Informer 패턴**: watch + 캐싱 + 이벤트 핸들러
- **재시도/백오프** 내장

4. 책 vs 현실



책(2018) → 현재(2024+)

항목	책	현재
SA 토큰	Secret 기반 영구 토큰	**Projected Token** (만료, 감사)
RBAC	"선택"	**기본 활성**
이미지	`luksa/kubectl-proxy:1.6.2`	`bitnami/kubectl:latest`
`tutum/curl`	권장	deprecated → `curlimages/curl:latest`
Downward 필드	책 시점 한정	+ `status.podIPs/hostIPs` (듀얼스택) + `hugepages-*`
Resource Resize	없음	**v1.35 stable** (volume 자동 갱신)

실습 메모: 부딪힌 이슈

- `memory: 4Mi` **에러**:
  `runc: Minimum memory limit allowed is 6MB` → **8Mi로 수정**
- **이미지 호환성**:
  `luksa/kubectl-proxy:1.6.2` (2018, k8s 1.8) 노후 → `bitnami/kubectl:latest`
- **labels 런타임 갱신 검증**:
  `kubectl label pod kubia env=test` → 5초 내 `/etc/downward/labels` 파일 변경 확인 ✅
- **RBAC 정확성**:
  Role에 pods만 → services 여전히 forbidden (Least Privilege 검증)

Key Takeaways

1. **Downward API**: 자기 메타데이터, env/volume 두 방식
2. **labels/annotations 전체**: env 불가, volume만
3. **단일 키** (`labels['app']`): env 가능
4. **API 직접 호출**: SA 토큰 + RBAC + (선택) Ambassador
5. **In-place resize**: env stale, volume 자동 갱신
6. **운영 도구는 Role을 정확히 좁혀** 부여 (Least Privilege)



Q & A

면접 예상 질문:

- env로 `metadata.labels` 받으려면?
- limits 없는데 `limits.cpu`를 노출하면?
- in-place resize 후 env / volume 동작 차이?
- Ambassador 패턴 vs 클라이언트 라이브러리?

